Authorability Without Prior: Measuring Whether Structured Tooling Replaces Training Data for a New Programming Language

Eduardo Díaz
ediaz@dcc8090.cl · October 2026 · second edition (first: July 2026)
Benchmark, harness, and full transcripts: github.com/lnds/kaikai-authorability-bench

Abstract. Large language models write mainstream programming languages from years of training data. A new language has none. We ask whether **structured tooling** — a machine-queryable language reference, typed holes, and diagnostics written to be acted upon — can substitute for that missing prior. We present an authorability benchmark for **kaikai**, a statically typed functional language with zero presence in any training corpus, evaluated head-to-head against Rust and Go across 294 agent-executed task cells: 15 tasks in two size tiers, three LLMs from two providers, and an enforced tooling ablation. Three findings. First, correctness saturates: every model matches its Rust/Go first-round success rate in kaikai (the two Claude models are green in round one in all 204 of their cells; the open-weight model's only failures were in the mainstream languages). Second, the missing prior is paid in **tokens**, not correctness: kaikai costs 4–15× the tokens of a known language, a multiple that grows as model strength drops and that does **not** shrink when tasks grow from single functions to modules. Third, the machine-queryable reference earns its keep: removing it raises the stronger Claude model's cost by 75%, even on tasks that barely touch the standard library. This edition re-runs the Claude grid on current models with agents isolated from persistent memory. Two findings of the first edition — that the token tax amortizes with task size, and that an available reference imposes a "curiosity tax" — did not survive that isolation and are withdrawn.

1 Introduction

A language model's fluency in a programming language is, to first order, a function of how much of that language it saw during training. This puts new languages in a cold-start bind: nobody writes them because models can't, and models can't because nobody writes them.

kaikai[1] is a statically typed functional language (HM inference, algebraic effects, fibers) designed with an explicit bet: that the cold-start can be broken with **tooling designed for LLM consumption** — a language reference queryable as JSON (`kai info --json`), typed holes that report expected types and in-scope bindings, and compiler diagnostics that state the repair, not just the error. The language's acceptance criteria encode the bet: *"an LLM with JSON access completes the top 80% of typical functions within one round of compilation."*

This paper reports the benchmark built to test that claim adversarially. The design point is asymmetry, declared up front: the models evaluated carry years of prior for Rust and Go and **zero** for kaikai — no documentation, no tutorials, no Stack Overflow answers exist in any training corpus. The benchmark therefore runs uphill for kaikai by construction. If kaikai matches the incumbents, the thesis is proven in its strongest form; where it loses, per-task transcripts say exactly which diagnostic or affordance failed, feeding the language's DX work queue.

2 Benchmark design

2.1 Protocol

Each **cell** is one task × language × seed. The agent receives the task specification, the exact target signature, and access to the language's standard toolchain inside an empty scratch directory (kaikai:

[1] https://github.com/lnds/kaikai

`kai run`, `kai info --json`, typed holes via `kai build --holes`; Rust: `rustc`; Go: `go build` / `go vet`). It delivers a solution file; the harness assembles it with **hidden tests** the agent never sees, compiles, and runs. On red, the compiler or test output is fed back verbatim and the agent tries again, up to five rounds. Everything is logged: rounds, tokens, model identity, full per-round transcripts.

Metrics: **success@1** (green on the first compile), **rounds-to-green** (compile attempts; agent infra errors such as timeouts are logged separately and do not count as attempts), and **tokens per cell** (total, including the agent's internal exploration turns).

Fairness controls: identical hidden tests adapted per language; the same prompt shape; no task requires O(1) random access as the only reasonable solution, so kaikai's cons-list idiom competes fairly with slice-based Rust and Go; every task ships a reference solution in all three languages, and a task enters the set only when all three references pass their hidden tests (a 195/195 smoke gate at time of writing).

Isolation. Every agent invocation starts from nothing: no persistent agent memory, no user or project instructions, and a scratch directory outside the benchmark repository, so neither earlier cells nor the reference solutions are reachable. The first edition did not enforce this for the Claude runs; Section 4 describes what that cost.

2.2 Tooling ablations, enforced

kaikai runs in four configurations: **full**, **no-info** (reference tooling unavailable), **no-holes**, and **flat-diags** (rich diagnostics reduced to traditional `file:line:col: error: message` one-liners). Ablations are **enforced**, not merely prompt-level: a generated `kai` shim on the agent's PATH rejects the ablated commands and, under flat-diags, filters every diagnostic the agent sees — its own probes and the harness feedback alike. This edition exercises **full** and **no-info**.

2.3 Tasks and models

The task set holds 65 tasks: 56 single-function and 9 multi-function modules balanced across three stdlib-surface classes (algorithm-heavy, record/state-heavy, string/stdlib-heavy). This edition reports the 15 that carry the load: the six-task edge-dense "hard" function subset (RFC 4648 Base64, ISO-8601 week numbering, strict CSV quoting, RPN evaluation with strict token rules, path normalization, interval scheduling) and all nine modules. Specifications are fully self-contained; hidden tests may demand nothing the signature does not declare.

Models: Claude Opus 5.5 and Claude Sonnet 5.5 (via the `claude` CLI, October 2026, kaikai 0.146.0), and GLM-5.2, an open-weight non-Anthropic model (via the `opencode` CLI, July 2026 on kaikai 0.99.x, carried over from the first edition) — the latter also validating that the harness protocol generalizes across agent CLIs. Each model runs every task in all three languages with 2 seeds; the Claude models additionally run the hard subset in kaikai under no-info. 294 valid cells total, none lost to agent infrastructure.

3 Results

3.1 Correctness saturates — for every model, in every language

model	tier	Go s@1	Rust s@1	kaikai s@1
Opus 5.5	functions	1.00	1.00	1.00
Opus 5.5	modules	1.00	1.00	1.00
Sonnet 5.5	functions	1.00	1.00	1.00
Sonnet 5.5	modules	1.00	1.00	1.00
GLM-5.2	functions	1.00	1.00	1.00
GLM-5.2	modules	0.94	0.94	**1.00**

Table 1: First-round success by model, tier, and language (full tooling configuration). Functions: 6 tasks × 2 seeds per language; modules: 9 × 2.

Across 294 cells, no language ever separated from another on correctness by more than one cell (Table 1). The Tier-3 acceptance claim asked for 80% of functions green in one round; the measured value is ≈100% — and it holds one tier up, at module scale, and across providers. Both Claude models were green in round one in every cell, the 24 no-info cells included. GLM-5.2 — open-weight, non-Anthropic, with zero kaikai prior — was **perfect in kaikai** (30/30 green@1) while its only two failures landed in Rust and Go, its training-prior languages. Two cells prove nothing alone; the sign, however, is the opposite of the null hypothesis, and it makes the thesis provider-independent rather than an artifact of one vendor's models.

The saturation itself is a finding: edge-dense specifications, exact-output contracts, and round-trip invariants do not dent current frontier models at function or module size. Correctness pressure for future iterations must come from repository-scale tasks or from kaikai-specific surface (effects and handlers) where mainstream idiom offers no crutch.

3.2 The missing prior is paid in tokens

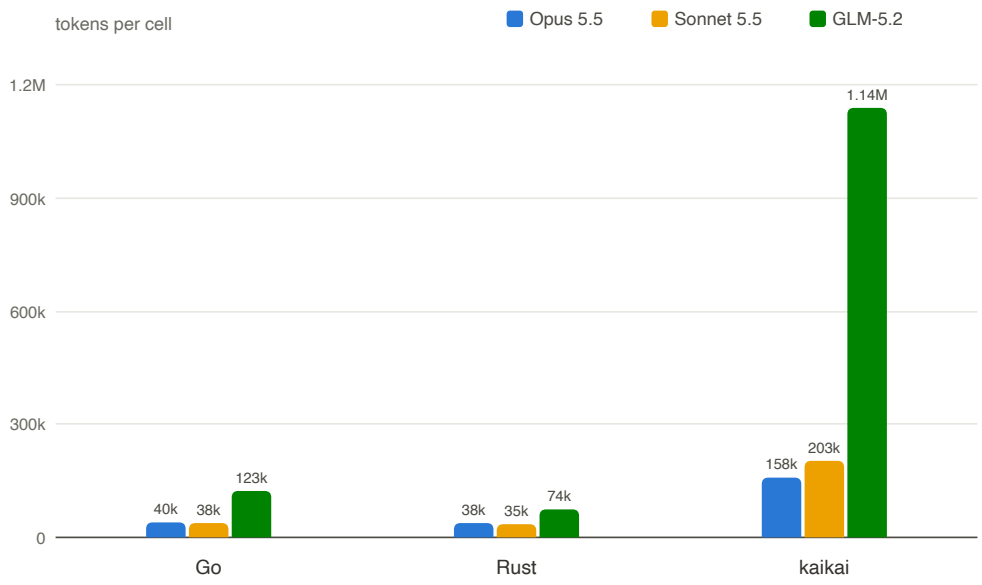


Figure 1: Tokens per cell on the hard-function batch, full configuration. kaikai costs a multiple of that model's cheaper mainstream language, and the multiple grows as the model tier drops: Opus 4.2×, Sonnet 5.8×, GLM-5.2 15.4×.

Correctness parity is not free (Figure 1). At function scale, writing kaikai costs Opus ≈158k tokens per cell against ≈38–40k for Rust/Go; Sonnet pays ≈203k against ≈35–38k; GLM-5.2 ≈1.14M against its ≈74–123k mainstream baseline. The missing prior is being purchased at run time — the agent reads the reference, probes with `kai run`, inspects holes — and **weaker models buy more of it**: a kaikai cell takes Opus ≈7 agent turns and Sonnet ≈9–11, where a Rust or Go cell takes either model ≈3.

Token counts include cached context re-reads (the dominant term), so these are utilization figures rather than dollar costs. As anchors: the 204 Claude cells cost $22 in total by the CLI's own accounting, a kaikai cell costing ≈2.6–3.1× a mainstream one in dollars against 4–7× in tokens; GLM-5.2′s 90 cells cost $59 at list prices. Absolute counts are not comparable across the two CLIs — each re-reads its own scaffolding on every turn — so the quantity to compare between models is the within-model multiple.

3.3 The tax does not amortize with task size

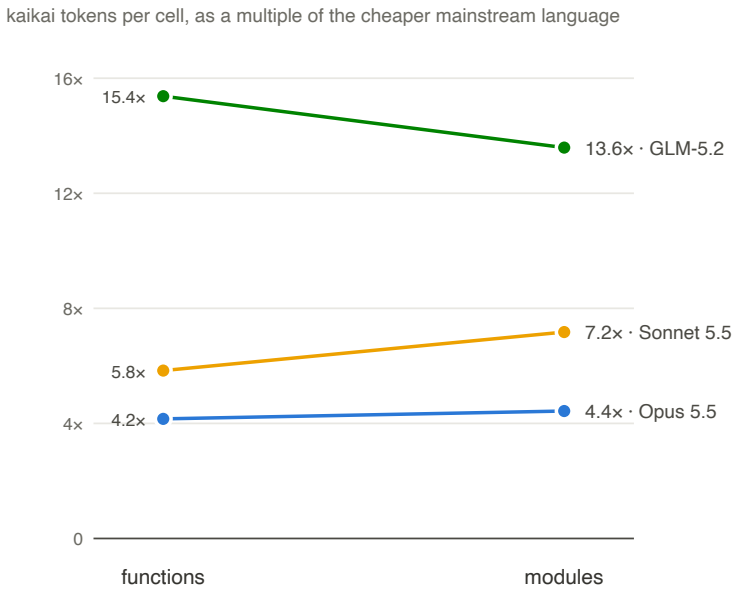


Figure 2: kaikai tokens per cell as a multiple of the same model's cheaper mainstream language, functions → modules (full configuration). The Claude multiples hold or rise; GLM-5.2′s dips only because its mainstream baseline grows — its kaikai cost is flat.

Moving from single functions to multi-function modules does not make the missing prior cheaper (Figure 2). kaikai's per-cell cost **rises** for both Claude models (Opus 158k→184k, +16%; Sonnet 203k→256k, +26%) while their mainstream languages move +2–9%, so the multiple holds or grows: Opus 4.2×→4.4×, Sonnet 5.8×→7.2×. GLM-5.2 is flat in absolute terms (1.14M→1.13M; 15.4×→13.6×). Every cell is a fresh session: what the agent learns about the language in one cell is gone in the next, so the exploration is re-paid per task, and a larger task asks for somewhat more of it.

This reverses the first edition, which reported a 47–58% drop for the Claude models and read it as within-session consolidation (Section 4). What remains open is the case that matters in practice — a long single session over a repository, where exploration genuinely could be spent across many edits. A per-cell design at these sizes cannot show it either way.

3.4 The reference pays for itself

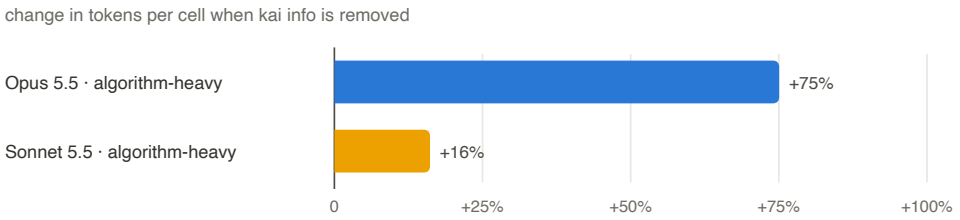


Figure 3: Enforced no-info ablation: change in kaikai tokens per cell on the hard-function batch when the machine-queryable reference (`kai info`, `kai doc`) is removed, relative to the full configuration on the same tasks (12 cells per bar).

Removing `kai info` makes kaikai more expensive, as the tooling thesis predicts (Figure 3). For Opus the effect is large and consistent: +75% tokens (158k→277k), ≈11 turns instead of ≈7, and a higher mean on each of the six tasks — the agent rediscovers syntax and APIs by trial-and-error compilation. For Sonnet the mean moves the same way (+16%, 203k→236k) but sits inside the seed-to-seed spread: three tasks cost more, three cost less. Neither model lost a single cell: all 24 no-info cells were still green in round one. At this size the reference buys **cost**, not correctness.

These are algorithm-heavy tasks — bit packing, calendar arithmetic, hand-rolled parsers — which barely touch the standard library; that the reference pays even here is the stronger form of the result. Stdlib-heavy tasks, where it should matter more, were not re-run for the current models, and neither were the no-holes and flat-diags configurations: with zero red rounds there is no feedback channel for rich diagnostics to improve, and their coefficients await tasks that actually fail.

For language design the reading is that reference consultation is where the tax is spent or saved, which makes its efficiency — symbol-sized answers rather than pages — the lever to pull[2].

3.5 What the red rounds contain

Among 294 cells, two reds were genuine model failures, both GLM-5.2′s and both in mainstream languages: an `Option::unwrap` panic in Rust and a failed assertion in Go, each green in the next round. The Claude models produced none in 204 cells; one Opus round under no-info timed out and was retried, which the protocol counts as agent infrastructure rather than a compile attempt.

The loop the language was designed around — prior collision → caught by compiler → explained by diagnostic → repaired — therefore went unexercised in this edition. The first edition recorded two such collisions in kaikai with the previous Claude generation (a `//` comment answered by a diagnostic naming `#`; an imperative accumulate-and-reverse idiom transplanted onto cons lists). Neither recurred.

4 What changed since the first edition

The first edition (July 2026) reported Claude Fable 5, Opus 4.8, and Sonnet 5. Preparing this one exposed a flaw in how those cells were run. The agent CLI keeps a persistent, per-project memory, and the scratch directories lived inside the benchmark repository, so every cell shared one memory store. Over the July runs the agents wrote 26 notes into it — kaikai syntax traps, standard-library names, and in one case the algorithm for a benchmark task — and every later cell started with the index of those notes in context. We verified the mechanism directly: an agent launched as in July reports the notes; launched under the isolation of this edition it does not.

[2]A tooling issue requesting exactly that was filed after the first edition (https://github.com/lnds/kaikai/issues/1166); its stated motivation was the curiosity-tax finding withdrawn here.

The July runs happened in a fixed order — hard functions under full tooling, then the no-info ablation, then modules — so memory accumulated in exactly the direction of two reported effects: modules cheaper than functions ("amortization"), and no-info cheaper than full (the "curiosity tax"). GLM-5.2, driven through a different CLI with no access to that store, showed neither, which the first edition interpreted as a difference in model capability.

With isolation and current models, neither effect appears, and both claims are withdrawn. This is not a controlled demonstration that memory produced the July pattern — the models and the kaikai release changed between editions too — but it removes the grounds for either claim. What is reproduced: correctness saturation, the existence of the token tax, and its ordering by model strength. The July Claude numbers remain in the repository's results log, marked as memory-contaminated.

One constructive reading survives. An agent that keeps notes about an unfamiliar language is a realistic deployment, not only a contaminant, and how fast the tax falls under accumulated memory is a legitimate measurement — provided it is designed as one, with the store's contents controlled and the run order randomized.

5 Discussion

The benchmark set out to measure whether structured tooling compensates for a missing training prior. On the **correctness** axis the compensation is total — indistinguishable from parity across two task tiers, three models, and two providers — which means the axis itself stops discriminating. What remains, and what discriminates in every experiment, is the **cost** axis: how many tokens a model spends conversing with the toolchain to reach the same green. "Designed for LLMs" is then not one coefficient but a family — cost by model tier, by tooling configuration — and the one tooling coefficient measured here says the structured reference is the part doing the work.

The result also suggests a practical adoption argument for new languages generally: the cold-start problem is real but it is a **pricing** problem, not a capability problem. A team choosing kaikai with a current Claude model pays roughly 4–7× in agent tokens — about 3× in dollars — at function and module scale alike, and none of it in correctness; with GLM-5.2 the multiple is ≈14–15×. Nothing measured here says the multiple shrinks as tasks grow, which turns reference-consultation efficiency from polish into the adoption bottleneck.

6 Threats to validity

Small samples per cell. Every grid runs 2 seeds per cell, 12 or 18 cells per row; the per-model coefficients carry meaningful variance, and Sonnet's +16% under no-info is not distinguishable from zero. **A subset of the tasks.** 15 of the 65 tasks are exercised, and the no-info ablation only on algorithm-heavy functions. **One task author.** All tasks were authored within this project; a pilot run caught one unfair hidden test (requiring undeclared trait derives in Rust), which was fixed — other biases may remain. **Token counts are utilization, not cost**, dominated by cached re-reads; one timed-out agent round contributes no tokens to its cell. **The two CLIs are not measured alike.** GLM-5.2′s cells date from July, on kaikai 0.99.x, with token counts recovered post-hoc from its CLI's local session store; they were not re-run under this edition's isolation protocol. That CLI has no access to the memory store that affected the July Claude runs, but its configuration was not audited to the same standard. **Saturation bounds the instrument.** With success@1 ≈ 1.0 everywhere, correctness comparisons are uninformative at these tiers for these models; claims are correspondingly about cost. **The asymmetry ages.** kaikai's absence from training corpora is what makes the experiment clean; the moment kaikai code (including this benchmark) enters a training set, the uphill condition erodes.

7 Related work

Code-generation benchmarks in the HumanEval/MBPP tradition measure correctness of single-shot completions in languages the models know well; SWE-bench moved the target to repository-scale edits, still within high-prior ecosystems. This benchmark differs on the independent variable: it holds the tasks fixed across languages and varies the **training prior and its tooling substitute**, using agents with compiler access rather than one-shot generation, and treating tokens-to-green as a first-class outcome rather than an implementation detail.

8 Conclusion

With structured tooling, having never existed in a training corpus turned out to be a cost problem, not a correctness problem. The cost has structure — it scales inversely with model strength, it is repaid on every task regardless of size, and the machine-queryable reference is what keeps it down. Two more elaborate claims from the first edition were artifacts of agents sharing memory across cells, and isolating the agents is now part of the protocol. The benchmark, its 65 tasks, the enforced ablation harness, and every transcript are public and re-runnable; each future kaikai release runs it again, and each red round becomes a language issue. The ratchet is the point.

Acknowledgments. Experiment orchestration, harness implementation, and task authoring were done in collaboration with Claude Code (Anthropic). GLM-5.2 was driven via the open-source `opencode` CLI.

References. Chen et al., *Evaluating Large Language Models Trained on Code* (HumanEval), arXiv:2107.03374, 2021. · Austin et al., *Program Synthesis with Large Language Models* (MBPP), arXiv:2108.07732, 2021. · Jimenez et al., *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*, arXiv:2310.06770, 2023. · kaikai: github.com/lnds/kaikai. · This benchmark: github.com/lnds/kaikai-authorability-bench.